Roadmap (revised) — Project 8: SDN steering of IoT traffic to the cloud

Data Communications group project · 2026–2027 · Group 442G · Members: Georgescu Mihai (A), Dragu Stefan (B), Nichita Herciu (C) · Repository: github.com/nherciu7/SDN-Steering · Version 2, November 2026 (simplified after teacher feedback)

1. Project and variant

Project: compare static routing with controller-driven (SDN) routing for IoT traffic that travels from a gateway site, across a small emulated WAN, to a cloud server. We measure latency, packet loss, throughput, the delay of the first packet, and (final) how the network recovers when a link fails.

Tools: Mininet with Open vSwitch (OpenFlow 1.3) and the Ryu controller (Python). Everything runs in the official Mininet VM in VirtualBox, the same image on all three laptops. Traffic is generated with standard tools: *ping* for IoT telemetry and *iperf3* for background load.

Topology: one IoT gateway switch (s1) with three hosts, two paths to the cloud switch (s4): a fast primary path through s2 (2 × 5 ms) and a slower backup path through s3 (2 × 15 ms). One cloud server (srv) is attached to s4. WAN links are limited to 10 Mbit/s so that background traffic has a visible effect. h1 and h2 are IoT sensors; h3 produces bulk traffic.

Changes from version 1: ONOS replaced with Ryu (lighter, Python, runs in the same VM); one gateway site and one data centre instead of two each; 4 switches instead of 6; 3 routing modes instead of 4 (the load-aware app is dropped); ping/iperf3 instead of custom generators; results reported as average with min/max over 3 repetitions instead of percentiles and confidence intervals.

2. Responsibilities (each member owns one part and can explain the whole project)

Member	Owns	Concrete deliverables
A — Georgescu Mihai Network & emulation	Mininet topology, link parameters, static routing, link failure	topology/topo.py (4 switches, 4 hosts, TCLink bandwidth/delay, fixed port numbers); topology/static_flows.sh (static flows with ovs-ofctl); link_failure.sh (take the primary link down during a run); CPU check.
B — Dragu Stefan Control plane	Ryu controller and the controller-driven modes	Ryu installed and connected; controller/reactive_app.py (installs the primary path on the first packet); controller/failover_app.py (switches to the backup path when a link goes down); controller log of installed flows.
C — Herciu Nichita Measurement & automation	Traffic, logging, KPI scripts, test runner, plots	measurement/run_test.py (starts topology + mode, runs ping and iperf3, saves logs); parse_results.py (logs → summary.csv); plot_results.py (bar charts); run_all.sh (whole matrix in one command); README with reproduction steps.

Shared: every member commits from their own GitHub account, writes the report section for their part and presents at midterm and final. Once a week we run all three parts together (integration test).

3. Experiment matrix

Block	Factors and levels	Fixed	Reps	Runs
E1 Mode × load (midterm: static, reactive)	Mode {static, reactive, failover} × background load {none, 5 Mbit/s iperf3}	60 s run, no failure	3	18
E2 Link failure (final)	Mode {static, reactive, failover}	Load none, primary link s1–s2 down at t = 20 s	3	9
Total				27

Each run takes about 90 s including Mininet start and clean-up, so the full matrix is about 40 minutes and is started with one command. All runs execute on one laptop, so all timestamps use the same clock. Every result in the report states the mode, the load, the number of repetitions and the run duration.

4. KPIs (one definition each in the report)

Round-trip time (ms, average / min / max) from ping between the sensors and the server; **packet loss** (%) from ping; **throughput** (Mbit/s) of the background flow from iperf3; **first-packet delay** (ms) = RTT of the first ping − average RTT of the remaining pings (shows the cost of asking the controller); **failover time** (ms, final) = number of consecutive lost pings × ping interval (0.1 s) after the link goes down.

5. Timeline

Weeks	Phase	Exit criterion	Lead
1–2	P0 Setup	Mininet VM on all laptops; Ryu installed; repository and roles	all
3–4	P1 Topology	topo.py runs; static flows give ping between all hosts and server	Georgescu

Weeks	Phase	Exit criterion	Lead
5–6	P2 Reactive + measurement	Reactive app works; test runner saves logs; parser gives summary.csv	Dragu, Herciu
7–8	**Midterm**	Demo of static and reactive; 12 runs (2 modes × 2 loads × 3 reps, 30 s); plots of RTT, first-packet delay and throughput; README	all
9–10	P3 Failover	Failover app and link-failure script work; failure test for each mode	Dragu, Georgescu
11–12	P4 Full matrix	27 runs complete; all plots generated by script	Herciu
13	Report	Report draft; README one-command reproduction	all
14	**Final**	Live demo, report, Q&A	all

6. Risks and fallbacks

Risk	Impact	Mitigation / fallback
Mininet does not run on Windows/WSL2 (no Open vSwitch module)	High	Use the official Mininet VM in VirtualBox on all three laptops.
Ryu fails to install on the VM's Python	Med	Pin compatible package versions; fallback: OS-Ken (maintained fork of Ryu, same API).
The topology has a loop (two paths), so flooding causes broadcast storms	Med	Static ARP in Mininet; no learning switch; flows match IPv4 destination only; switches in secure mode.
Laptop CPU limits iperf3 throughput	Low	Small topology, rate-limited iperf3 (5 Mbit/s, 50 % of the link).
Static mode does not recover after link failure	Low	Expected behaviour; reported as a result.
Parts do not fit together	Med	Fixed topology spec (names, IPs, port numbers) agreed in the repository; weekly integration test.

Extension (only if the core is done): give IoT telemetry priority over bulk traffic with an OVS priority queue and measure the RTT difference under load.

AI-assistance declaration: AI (Claude) was used to help write and format this roadmap and as a coding assistant; every member reviews, tests and commits their own part and can explain it.